

Python Interview Questions

1. Python Libraries Commonly Used by Data Analysts

Data analysts frequently use these Python libraries:

- **pandas** – Data manipulation and analysis using Data Frames.
- **numpy** – Numerical operations and handling large arrays.
- **matplotlib** – Basic data visualization (line plots, bar charts, histograms, scatter plots).
- **seaborn** – Statistical visualization (heatmaps, box plots, violin plots).
- **scipy** – Scientific computing (statistical analysis, optimization).
- **scikit-learn** – Machine learning (classification, regression, clustering).
- **statsmodels** – Statistical modeling and hypothesis testing.
- **plotly** – Interactive visualizations and dashboards.
- **openpyxl** – Reading and writing Excel files.
- **requests** – Fetching data from web APIs.

2. Difference Between Lists, Tuples, and Dictionaries in Python

Feature	List	Tuple	Dictionary
Definition	Ordered, mutable collection	Ordered, immutable collection	Unordered key-value pairs
Mutability	Can be modified	Cannot be modified	Values can change, keys must be immutable
Indexing	Supports indexing/slicing	Supports indexing/slicing	Access via keys
Performance	Slower due to mutability	Faster due to immutability	Fast lookup (hash table)
Use Case	Dynamic collection (e.g., names list)	Fixed collection (e.g., coordinates)	Fast key-based lookup (e.g., customer data)

3. Handling Missing Data in a pandas DataFrame

- **Identify missing data**
`df.isnull().sum()` → Counts missing values in each column
- **Fill missing values**
 - With **mean**: `df['Sales'].fillna(df['Sales'].mean(), inplace=True)`
 - With **median**: `df['Sales'].fillna(df['Sales'].median(), inplace=True)`
 - With **forward fill (ffill)**: `df.fillna(method='ffill', inplace=True)`
 - With **backward fill (bfill)**: `df.fillna(method='bfill', inplace=True)`
- **Drop missing rows**: `df.dropna(inplace=True)`

Example Scenario: If a UK retail sales dataset has missing Revenue values, using the median ensures robustness against outliers.

4. Reading a CSV File into a pandas DataFrame and Inspecting First Few Rows

- **Read CSV file**
`df = pd.read_csv("customer_data.csv")`
- **Inspect first few rows**
`df.head()`
- **Check structure**
`df.info()` → Column types and missing values
`df.describe()` → Summary statistics

Example Scenario: Loading `customer_data.csv` helps analyze customer demographics and purchase history.

5. Difference Between `.loc[]` and `.iloc[]` in pandas

Feature	<code>.loc[]</code>	<code>.iloc[]</code>
Indexing Type	Label-based	Integer-based
Usage	Uses row/column labels	Uses numerical positions
Example	<code>df.loc[2, "Revenue"]</code> → Row index 2, column "Revenue"	<code>df.iloc[2, 3]</code> → Row position 2, column position 3
Slicing	Inclusive	Exclusive

When to Use:

- Use `.loc[]` for named indices/labels.
- Use `.iloc[]` for strict positional selection.

6. Filtering Rows in a pandas DataFrame Based on a Condition

Filtering allows selecting specific rows based on conditions.

Example Scenario:

You have a dataset containing customer transactions, and you want to **select only transactions above £500**.

How to filter rows?

- To select transactions where the amount is greater than **£500**:
`df_filtered = df[df['TransactionAmount'] > 500]`
- To filter **multiple conditions** (e.g., transactions above £500 in London):
`df_filtered = df[(df['TransactionAmount'] > 500) & (df['City'] == 'London')]`

This helps in **identifying high-value sales** or **analyzing premium customer behavior**.

7. Purpose of the groupby Function in pandas

The `groupby()` function is used to **group data based on a column** and apply an **aggregate function** like `sum`, `mean`, or `count`.

Example Scenario:

You have a UK sales dataset and need to **summarize total sales by region**.

How to group and summarize data?

- To get **total sales per region**:
`df_grouped = df.groupby('Region')['Sales'].sum().reset_index()`
- Other useful group operations:
 - To get **average sales per region**: `df.groupby('Region')['Sales'].mean()`
 - To count **number of transactions per region**:
`df.groupby('Region')['Sales'].count()`

This helps **compare performance across different regions** and identify **which areas generate the most revenue**.

8. Merging or Joining Two DataFrames in pandas

Merging combines two datasets based on a **common column (key column)**.

Example Scenario:

You have two datasets:

1. **Customer Demographics** (CustomerID, Name, Age, Gender)
2. **Purchase History** (CustomerID, TransactionAmount, Date)

To analyze **customer purchase behavior along with their demographics**, you need to **merge both datasets** using CustomerID.

How to merge DataFrames?

- To merge customer data with purchase history:
`df_merged = pd.merge(df_customers, df_purchases, on='CustomerID', how='inner')`

Types of joins:

- **Inner Join:** Only keeps customers who **appear in both datasets** (how='inner').
- **Left Join:** Keeps **all customers** from demographics, even if they **don't have a purchase** (how='left').
- **Right Join:** Keeps **all purchases**, even if **some customers are missing** (how='right').
- **Outer Join:** Keeps **all records from both datasets**, filling missing values with NaN (how='outer').

This merging allows creating a **detailed customer profile**, combining both personal details and purchase behavior.

9. What are Lambda Functions, and How Are They Used in Python?

Lambda functions are **anonymous functions** used for short operations in a **single line**.

Example Scenario:

You need to **apply a custom transformation** to a column in a DataFrame, such as **applying a discount to sales prices**.

How to use lambda functions?

- To **apply a 10% discount** to all product prices:
`df['DiscountedPrice'] = df['Price'].apply(lambda x: x * 0.9)`
- More **lambda function examples**:
 - Convert **names to uppercase**:
`df['CustomerName'] = df['CustomerName'].apply(lambda x: x.upper())`
 - Categorize **sales into "High" or "Low"**:
`df['Category'] = df['Sales'].apply(lambda x: 'High' if x > 1000 else 'Low')`

Lambda functions are **efficient for quick transformations** without defining a separate function.

10. Creating a Line Plot Using Matplotlib to Show Sales Trends Over Time

A **line plot** is useful for **visualizing trends over time**, such as **sales performance across months**.

Example Scenario:

You have a dataset with **daily sales records**, and you want to **analyze sales trends over time**.

How to create a line plot?

1. Basic line plot for sales trends:

- `plt.plot(df['Date'], df['Sales'], marker='o', linestyle='-')`
- `plt.xlabel('Date')`
- `plt.ylabel('Sales')`
- `plt.title('Sales Trends Over Time')`
- `plt.xticks(rotation=45)`
- `plt.show()`

2. Plot multiple sales lines for different regions:

- Loop through unique regions and plot sales for each.
- Add **labels and legends** to differentiate the regions.

This helps in **identifying sales patterns, seasonal fluctuations, and growth trends**.

11. Using the `apply()` Function in Pandas for Column-Wise Transformations

The `apply()` function in pandas is used to apply a custom function to each element of a column or row.

Example Scenario:

You have a dataset with product prices in **different currencies**, and you need to **standardize all prices to GBP (£)**.

How to Use `apply()` for Currency Conversion?

1. Suppose your dataset has a **column named "Price_USD"**, and you want to convert it to GBP using an exchange rate of **1 USD = 0.75 GBP**.

2. You can use `apply()` to transform all values:

```
df['Price_GBP'] = df['Price_USD'].apply(lambda x: x * 0.75)
```

Other Use Cases of `apply()`

- **Standardizing numerical data (e.g., normalizing between 0 and 1):**

```
df['NormalizedPrice'] = df['Price'].apply(lambda x: (x - df['Price'].min()) /  
(df['Price'].max() - df['Price'].min()))
```

- **Formatting strings (e.g., making all product names lowercase):**

```
df['ProductName'] = df['ProductName'].apply(lambda x: x.lower())
```

Using `apply()` helps in **quickly transforming column data** without writing complex loops.

12. Purpose of the `pivot_table()` Function in Pandas

A pivot table summarizes data **by aggregating values based on categories**.

Example Scenario:

You have a dataset containing **monthly sales from different regions**, and you want to create a **summary table showing total sales per region per month**.

How to Create a Pivot Table?

- If your dataset has columns: **Date, Region, and Sales**, you can use:

```
df_pivot = df.pivot_table(values='Sales', index='Month', columns='Region',  
aggfunc='sum')
```

Breaking It Down:

- **values='Sales'** → The column you want to summarize.
- **index='Month'** → Groups data by month.
- **columns='Region'** → Creates separate columns for each region.
- **aggfunc='sum'** → Aggregates data by summing up sales.

Why Use `pivot_table()`?

- Helps summarize large datasets into a structured view.
- Enables **quick comparison** of sales across months and regions.
- Supports various aggregation functions like **sum, mean, count, min, max**.

13. Difference Between Wide and Long Data Formats & Reshaping Data Using Pandas

In data analysis, datasets can be stored in **wide format** or **long format**.

Wide Format vs. Long Format

Format	Description	Example
Wide Format	Each column represents a category, and each row represents a subject.	Sales data with separate columns for each month.
Long Format	Data is stored in a stacked format , with multiple rows per subject.	Sales data with a single "Month" column and corresponding sales value.

Example

Wide Format (Before Reshaping)	Long Format (After Reshaping)
Product	Jan Sales
Laptop	1000
Phone	800

Reshaping from Wide to Long Format

To reshape data from **wide to long**, use `melt()`:

```
df_long = df.melt(id_vars=['Product'], var_name='Month', value_name='Sales')
```

Reshaping from Long to Wide Format

To reshape data from **long to wide**, use `pivot()`: `df_wide = df.pivot(index='Product', columns='Month', values='Sales')`

Reshaping is essential for preparing data for visualization and analysis.

14. Calculating Correlation Between Two Variables in Python

Correlation measures **the relationship between two variables**.

Example Scenario:

You want to **analyze the impact of marketing spend on sales** in the UK.

How to Calculate Correlation?

1. If your dataset has "**MarketingSpend**" and "**Sales**" columns, use: `correlation = df['MarketingSpend'].corr(df['Sales'])`
2. This returns a **correlation coefficient (r-value)**:
 - **r = 1** → Perfect positive correlation (more marketing = more sales).

- $r = -1$ → Perfect negative correlation (more marketing = fewer sales).
- $r = 0$ → No correlation (marketing spend doesn't affect sales).

Interpreting Results

- If $r > 0.5$, marketing spend is **strongly** influencing sales.
- If $r < 0.3$, the relationship is **weak or insignificant**.
- If r is **negative**, more marketing **reduces sales** (could be due to poor strategy).

This analysis **helps businesses optimize marketing budgets**.

15. Purpose of the datetime Module & Using It in Data Analysis

The datetime module helps in **handling date and time data** in Python.

Example Scenario:

You have transaction timestamps, but you want to analyze **daily sales trends**.

How to Use datetime in Data Analysis?

1. **Convert a column to datetime format:**
`df['TransactionDate'] = pd.to_datetime(df['TransactionDate'])`
2. **Extract specific time components:**
 - Extract **year**: `df['Year'] = df['TransactionDate'].dt.year`
 - Extract **month**: `df['Month'] = df['TransactionDate'].dt.month`
 - Extract **day of the week**: `df['Weekday'] = df['TransactionDate'].dt.day_name()`
3. **Group sales data by date to analyze daily trends:**
`daily_sales = df.groupby(df['TransactionDate'].dt.date)['Sales'].sum()`

Why Use datetime?

- Helps **convert timestamps into meaningful time units**.
- Enables **time-based trend analysis** (e.g., daily, monthly sales patterns).
- Useful for **time series forecasting and business insights**.

16. Handling Outliers in a Dataset Using Python

Outliers are extreme values that **deviate significantly from the majority of the data**. They can **skew analysis results** and impact model performance.

Example Scenario: Removing Revenue Anomalies in a UK Retail Dataset

You have a **revenue dataset**, but some transactions have **unusually high or low values**, which may be due to **errors or fraud**.

Methods to Handle Outliers:

1. **Detect Outliers Using the Interquartile Range (IQR) Method:**
 - Calculate **Q1 (25th percentile) and Q3 (75th percentile)**.
 - Compute **IQR = Q3 - Q1** (interquartile range).
 - Define outlier threshold: Values **below Q1 - 1.5 * IQR** or **above Q3 + 1.5 * IQR** are outliers.
 - Remove outliers: `df = df[(df['Revenue'] >= lower_bound) & (df['Revenue'] <= upper_bound)]`
2. **Use Z-Score to Identify Extreme Values:**
 - Compute **Z-score** = (value - mean) / standard deviation.
 - Remove data points with **Z-score > 3 or < -3**.
3. **Capping/Flooring Outliers (Winsorization):**
 - Instead of **removing**, replace extreme values with **upper and lower limits**.
 - Helps **preserve the dataset size** while reducing extreme impact.
4. **Visualizing Outliers:**
 - Use **boxplots (seaborn.boxplot())** or **histograms** to detect anomalies.

By handling outliers properly, you ensure **reliable data analysis and accurate insights**.

17. Difference Between merge, concat, and join in Pandas

Pandas provides multiple ways to **combine DataFrames**, each suited for different scenarios.

Method	Description	Use Case
merge()	Joins DataFrames on a common key column (like SQL join).	Combining customer data (CustomerID) with purchase history.
concat()	Stacks DataFrames vertically or horizontally .	Appending monthly sales data across multiple files.
join()	Joins DataFrames using index values instead of a key column.	Merging datasets with the same index (e.g., time-series data).

When to Use Each?

- Use **merge()** when working with **relational datasets** (e.g., combining customer details with transactions).
- Use **concat()** when you have **datasets with the same structure** and need to **stack them together**.
- Use **join()** when merging datasets **based on the index** instead of a column.

Example:

- `pd.merge(df_customers, df_orders, on='CustomerID', how='inner')` → Matches **customer orders** with **customer details**.
- `pd.concat([df_sales_Jan, df_sales_Feb], axis=0)` → Stacks **January and February sales** into one dataset.
- `df1.join(df2, how='left')` → Joins **time-series data** where both DataFrames share the same index.

Understanding these methods helps in **efficiently combining datasets** based on structure and requirements.

18. Automating Repetitive Data Cleaning Tasks Using Python

Data cleaning is often repetitive, so **automation saves time and ensures consistency**.

Example Scenario: Writing a Script to Clean and Format Monthly Sales Data

Imagine every month you receive a **sales dataset** that needs the following cleaning steps:

1. **Remove missing values.**
2. **Convert date columns to proper format.**
3. **Standardize column names.**
4. **Remove outliers from sales amounts.**
5. **Save the cleaned data for analysis.**

Steps to Automate Data Cleaning:

1. **Define a Function to Clean Data**
 - Write a function that takes a DataFrame and **applies all cleaning steps**.
2. **Use a Loop to Process Multiple Files**

- If you receive **monthly CSV files**, automate reading and cleaning them in a loop.

3. Save the Cleaned Data Automatically

- After cleaning, save the **processed files into a new directory**.

Why Automate?

- Saves time by **avoiding repetitive manual cleaning**.
- Ensures **consistent data quality** every time.
- Can be scheduled to **run automatically** when new data arrives.

19. Purpose of the Seaborn Library and Difference from Matplotlib

Seaborn is built on **Matplotlib** and specializes in **statistical data visualization**.

Key Differences Between Seaborn and Matplotlib

Feature	Seaborn	Matplotlib
Purpose	High-level statistical visualization	Low-level flexible plotting
Customization	Beautiful default themes	More customization needed
Best for	Complex visualizations (e.g., heatmaps, pairplots)	Simple plots (e.g., bar charts, line graphs)

Example Scenario: Visualizing Customer Segmentation Using a Pairplot

You have **customer segmentation data** (Age, Income, Spending Score), and you want to visualize the relationships.

- **Using Seaborn's pairplot()** → Automatically plots scatter plots for multiple numerical columns.
- `seaborn.pairplot(df, hue='Segment')` → **Color codes each segment** in the scatter plots.

When to Use Seaborn?

- For **statistical plots** (e.g., correlations, clustering, distributions).
- When you need **aesthetically pleasing** plots with minimal code.

Seaborn simplifies **complex visualizations** that require **more effort in Matplotlib**.

20. Detecting and Replacing Missing or Invalid Data in a Dataset Using Python

Missing or invalid data can **distort analysis** if not handled properly.

Common Methods to Handle Missing Data:

1. Identify Missing Data:

- `df.isnull().sum()` → Counts missing values per column.

2. Replace Missing Data:

- **Fill with the mean:** `df['Sales'].fillna(df['Sales'].mean(), inplace=True)`
- **Fill with the median:** `df['Sales'].fillna(df['Sales'].median(), inplace=True)`
- **Forward fill (use last valid value):** `df.fillna(method='ffill', inplace=True)`
- **Backward fill (use next valid value):** `df.fillna(method='bfill', inplace=True)`

3. Drop Rows with Missing Data (if necessary):

- `df.dropna(inplace=True)`

4. Replace Invalid Data:

- **Example:** Convert invalid negative sales values to NaN:
`df['Sales'] = df['Sales'].apply(lambda x: x if x >= 0 else None)`

Example Scenario: Cleaning a Dataset with Missing and Invalid Values

You have a dataset with **missing sales data** and some negative values (which are incorrect).

1. **Replace negative values with NaN.**
2. **Fill missing values using the median.**
3. **Remove rows with too many missing values.**

By detecting and replacing missing/invalid data, you **improve data quality and analysis accuracy**.

21. Identifying the Top-Selling Product Category for Each Region in the UK

When dealing with **thousands of rows of sales and customer data**, you need to determine which product category generates the highest sales in **each region**.

Steps to Identify the Top-Selling Product Category by Region

1. Ensure the dataset has key columns:

- **Region** → Represents different UK regions.
- **ProductCategory** → Groups products into categories (e.g., Electronics, Clothing).

- Sales → Total revenue per transaction.
- 2. **Group data by Region and ProductCategory using groupby():**
 - Compute **total sales per category per region** using sum().
- 3. **Use idxmax() to find the highest-selling product category for each region.**
 - This finds the category with the highest total sales in each region.

Example Output

Region	Top Product Category	Total Sales (£)
London	Electronics	500,000
Manchester	Clothing	320,000
Birmingham	Home & Garden	280,000

Why This Approach Works?

- groupby() efficiently **aggregates data at the category level**.
- idxmax() quickly finds the **top category per region**.
- **Scales well** even if the dataset has **millions of rows**.

This method allows **retail managers to see which product types perform best in each region**, helping in **marketing and inventory planning**.

22. Creating a Monthly Revenue Trend Report

To **analyze revenue trends over time**, we must:

- **Aggregate sales by month** using pandas groupby().
- **Visualize trends** using matplotlib.

Steps to Prepare the Report

1. **Ensure dataset has a Date column in datetime format.**
 - Convert it using pd.to_datetime().
2. **Extract the month and year for grouping.**
 - Use dt.to_period('M') to create a Year-Month column.
3. **Group sales data by Year-Month and calculate total revenue.**
 - df.groupby('Year-Month')['Revenue'].sum()

4. Plot revenue trends using matplotlib (line chart).

- X-axis: Month-Year
- Y-axis: Total Revenue (£)
- Use **markers to highlight trends**.

Why This Works?

- **Grouping by month** summarizes revenue efficiently.
- **Line charts highlight trends over time** (e.g., seasonal spikes in holiday months).
- Helps **businesses forecast sales trends and adjust marketing strategies**.

23. Consolidating and Analyzing Multiple Excel Files Containing Quarterly Sales Data

Your manager provides **multiple Excel files (one per quarter)**. You need to **merge them into a single dataset and analyze trends**.

Steps to Consolidate and Analyze Quarterly Data

1. **Import all Excel files dynamically** using `glob.glob()`.
 - If files are named **Q1_sales.xlsx, Q2_sales.xlsx**, use `glob('*.xlsx')` to read them dynamically.
2. **Read and concatenate all files into one DataFrame** using `pd.concat()`.
 - Ensures all quarterly sales data is **stacked together**.
3. **Ensure consistency in column names and data types**.
 - Standardize date formats and numerical columns.
4. **Perform analysis:**
 - **Total revenue per quarter:** `df.groupby('Quarter')['Sales'].sum()`
 - **Best-selling products per quarter.**
 - **Compare revenue growth over quarters.**
5. **Save the consolidated dataset for further analysis.**
 - `df.to_csv('Annual_Sales_Report.csv', index=False)`

Why This Approach Works?

- **Efficiently merges multiple files** into a single dataset.
- **Automates quarterly analysis** instead of manual merging.
- **Helps businesses see sales trends and seasonal patterns.**

24. Handling Missing Values in Critical Columns Like Revenue and Product Category

Missing values in **key columns** (e.g., Revenue, ProductCategory) can **impact decision-making**.

How to Handle Missing Values?

1. Check missing data patterns:

- `df.isnull().sum()` → Identifies missing values in each column.

2. Strategy for Numeric Columns (Revenue)

- If a few values are missing, use **median filling**:
`df['Revenue'].fillna(df['Revenue'].median(), inplace=True)`
- If **many values are missing**, explore **predictive imputation** (e.g., using machine learning).

3. Strategy for Categorical Columns (ProductCategory)

- Fill with **mode** (most frequent category):
`df['ProductCategory'].fillna(df['ProductCategory'].mode()[0], inplace=True)`
- If missing **too frequently**, consider **group-wise filling**:
`df['ProductCategory'] =
df.groupby('Region')['ProductCategory'].transform(lambda x:
x.fillna(x.mode()[0]))`

4. For completely unreliable data, drop rows:

- If missing values are **too large (>30%)**, consider dropping those rows:
`df.dropna(subset=['Revenue', 'ProductCategory'], inplace=True)`

Why This Approach Works?

- **Prevents data loss** by filling missing values logically.
- **Ensures business insights remain accurate.**
- **Uses different methods for numeric vs categorical data.**

25. Creating an Interactive Dashboard Using Python

Stakeholders want a **clear and interactive dashboard** to understand **customer trends and sales performance**.

Steps to Create an Interactive Dashboard

1. **Choose a Python visualization framework:**
 - **Use Plotly or Dash** for interactive web-based dashboards.
2. **Load the customer dataset into a Pandas DataFrame.**
 - Ensure key metrics are available: CustomerRegion, Sales, ProductCategory, etc.
3. **Create Interactive Charts Using Plotly:**
 - **Bar Chart:** Top-selling product categories.
 - **Line Chart:** Revenue trends over time.
 - **Heatmap:** Sales by region and product category.
4. **Add Interactivity Using Dash:**
 - Dropdowns for **filtering by region**.
 - Date selectors for **viewing time-based sales trends**.
 - Clickable elements for **drilling down into details**.
5. **Deploy the dashboard for stakeholders.**
 - Use **Dash on a web server** or **streamlit** for quick deployment.

Why Use Dash or Plotly?

- **Dynamic & Interactive:** Users can **filter, zoom, and hover** over data points.
- **Web-Based:** Accessible via a web browser.
- **No Need for Excel Reports:** Automated insights instead of static spreadsheets.

Task	Solution
Top-Selling Product Category by Region	Use <code>groupby()</code> , <code>sum()</code> , and <code>idxmax()</code> to find the highest revenue product category per region.
Monthly Revenue Trend Report	Use <code>groupby()</code> for aggregation and <code>matplotlib</code> for visualization (line chart).
Consolidating Quarterly Excel Data	Read multiple Excel files, concatenate them, and analyze sales trends per quarter.
Handling Missing Values	Use <code>fillna()</code> with median for numerical columns and mode for categorical columns .
Creating an Interactive Dashboard	Use Plotly/Dash to build dynamic visualizations with filters and charts.